



SIMONE DE BEAUVOIR
ÉCOLE FRANÇAISE
FONDATION NOUR EL MAAREF - 1992

1ère
NSI

Chapitre 3 : Traitement de données en table

Leçon 3 : Tri de table

Thème : Traitement de données en table

Enseignant : Housseem Lahiani

Introduction

Cette leçon explique comment trier une table de données, en particulier en utilisant le langage de programmation Python et sa bibliothèque standard ou la bibliothèque externe pandas pour manipuler des fichiers CSV. Voici un résumé de la leçon :

Trier une Table

Trier une table signifie organiser ses données en fonction des valeurs d'une ou plusieurs de ses colonnes. On peut accomplir cela de deux manières principales :

1. Sans utiliser pandas : On charge le fichier CSV dans une structure de données Python (comme une liste de listes ou une liste de dictionnaires) et on écrit une fonction pour effectuer le tri.
2. Avec pandas : On utilise la bibliothèque pandas, qui offre des outils puissants et simples pour trier les données.

La commande `sorted`

- La commande `sorted` permet de trier des listes ou des dictionnaires en Python.
- Contrairement à la méthode `sort()` qui modifie la liste sur laquelle elle est appliquée, `sorted` renvoie une nouvelle liste triée sans modifier l'original.
- Exemples d'utilisation de `sorted` sur des listes, des dictionnaires et des tuples sont fournis pour illustrer comment trier selon différents critères, notamment en utilisant une fonction lambda ou l'outil `itemgetter` pour les tris plus complexes.

Voici quelques exemples supplémentaires pour illustrer l'utilisation de la fonction `sorted` et de la méthode `sort` en Python, en travaillant avec différents types de données :

```
### Tri d'une liste de nombres dans l'ordre décroissant
```

```
nombres = [1, 7, 4, 2, 9]
```

```
nombres_tries = sorted(nombres, reverse=True)
```

```
print(nombres_tries) # Affiche [9, 7, 4, 2, 1]
```

Tri d'une liste de chaînes de caractères par longueur

```
mots = ["pomme", "banane", "cerise", "datte"]
```

```
mots_tries = sorted(mots, key=len)
```

```
print(mots_tries) # Affiche ['datte', 'pomme', 'banane', 'cerise']
```

Lorsque la fonction `sorted()` trie les éléments d'une liste et que plusieurs éléments ont la même clé de tri (dans ce cas, la même longueur de mot), elle maintient leur ordre relatif initial. Cela signifie que si deux mots ont la même longueur, l'ordre dans lequel ils apparaissent dans la liste d'origine est conservé dans la liste triée. Ce comportement est connu sous le nom de stabilité du tri.

Tri d'une liste de tuples par le second élément

```
tuples = [(1, 'pomme'), (2, 'banane'), (3, 'cerise')]
```

```
tuples_tries = sorted(tuples, key=lambda x: x[1])
```

```
print(tuples_tries) # Affiche [(2, 'banane'), (3, 'cerise'), (1, 'pomme')]
```

Utilisation de `sort` pour trier une liste d'entiers sur place

```
entiers = [5, 2, 3, 1, 4]
```

```
entiers.sort()
```

```
print(entiers) # Affiche [1, 2, 3, 4, 5]
```

Tri d'une liste de dictionnaires par une clé spécifique

```
dictionnaires = [{'nom': 'Alice', 'age': 30}, {'nom': 'Bob', 'age': 25},  
{ 'nom': 'Charlie', 'age': 35}]  
  
dictionnaires_tries = sorted(dictionnaires, key=lambda x: x['age'])  
  
print(dictionnaires_tries)  # Affiche [{'nom': 'Bob', 'age': 25}, {'nom':  
'Alice', 'age': 30}, {'nom': 'Charlie', 'age': 35}]
```

L'expression `key=lambda x: x['age']` utilisée dans la fonction `sorted` ou la méthode `sort` en Python sert à définir un critère de tri personnalisé. Expliquons chaque élément pour clarifier :

La fonction lambda

Une fonction `lambda` est une petite fonction anonyme en Python.

Elle peut prendre n'importe quel nombre d'arguments, mais ne peut avoir qu'une seule expression.

La syntaxe d'une fonction `lambda` est : `lambda arguments: expression`.

L'expression est évaluée et renvoyée lors de l'appel de la fonction `lambda`.

```
key=lambda x: x['age']
```

Dans le contexte de `sorted` ou `sort`, l'argument `key` spécifie une fonction à utiliser pour chaque élément de la liste (ou de tout autre itérable) sur lequel `sorted` ou `sort` est appelé.

Cette fonction retourne une valeur qui sera utilisée comme base pour le tri.

En utilisant `key=lambda x: x['age']`, vous dites essentiellement : "Pour chaque élément `x` dans la liste, utilisez `x['age']` comme clé de tri."

Cela signifie que la liste sera triée en fonction des valeurs associées à la clé `'age'` dans chaque dictionnaire de la liste.

Tri insensible à la casse d'une liste de mots

```
mots = ["Banane", "abricot", "Cerise", "datte"]

mots_tries = sorted(mots, key=str.lower)

print(mots_tries) # Affiche ['abricot', 'Banane', 'Cerise', 'datte']
```

Le paramètre `key=str.lower` est utilisé pour spécifier une fonction (dans ce cas, `str.lower`) qui est appliquée à chaque élément de la liste avant de procéder au tri. La fonction `str.lower` convertit une chaîne de caractères en minuscules, ce qui permet de comparer les éléments de manière équivalente, peu importe leur casse initiale. Ces exemples montrent la flexibilité de la fonction `sorted` et de la méthode `sort` pour trier divers types de données en Python, en utilisant des critères de tri personnalisés tels que la longueur, un élément spécifique d'un tuple, une clé dans un dictionnaire, ou même en rendant le tri insensible à la casse.

Trier un fichier CSV

- On peut trier un fichier CSV directement en utilisant `sorted`, en ouvrant le fichier, en le lisant et en triant les données selon une colonne spécifique avant d'écrire le résultat trié dans un nouveau fichier.
- Des exemples montrent comment réaliser cette opération soit en traitant les données comme une liste de listes (sans en-tête) soit comme une liste de dictionnaires (avec en-tête).

Supposons que nous avons un fichier csv qui contient ceci :

```
Nom,Age,Ville
Alice,25,Paris
Bob,30,New York
Claire,28,Sydney
```

Nous allons lire et trier un fichier CSV contenant des données sous la forme "Nom,Age,Ville" sans utiliser la fonction `next()` pour sauter l'en-tête. Nous ferons cela en utilisant le module `csv` de Python pour lire les données, puis nous utiliserons la fonction `sorted()` pour les trier selon un critère spécifique, par exemple par âge.

Étapes

- Ouvrir et lire le fichier CSV : Utiliser `csv.reader` pour lire le fichier. Nous lirons toutes les lignes, y compris l'en-tête.
- Trier les données : Trier les données en utilisant `sorted()`. Nous devons être prudents pour ne pas trier l'en-tête avec les données.
- Afficher ou sauvegarder le résultat : Après avoir trié, nous pouvons afficher les données triées

```
import csv
```

```
# Ouvrir et lire le fichier CSV
```

```
with open('donnees.csv', 'r') as fichier:
```

```
    lecteur_csv = csv.reader(fichier)
```

```
    # Conversion en liste pour faciliter le traitement
```

```
    donnees = list(lecteur_csv)
```

```
    # Séparer l'en-tête des données
```

```
    en_tete = donnees[0]
```

```
    donnees_sans_entete = donnees[1:]
```

```
    # Trier les données par âge (élément d'index 1)
```

```
    donnees_triees = sorted(donnees_sans_entete, key=lambda x: int(x[1]))
```

```
    # Afficher les résultats
```

```
    print(en_tete)  # Affiche l'en-tête
```

```
    for ligne in donnees_triees:
```

```
        print(ligne)
```

même chose mais avec utilisation de DictReader

Pour traiter des données CSV comme une liste de dictionnaires tout en conservant l'en-tête pour les clés des dictionnaires, vous pouvez utiliser le module ``csv`` de Python avec ``csv.DictReader``. Cela crée un objet qui fonctionne comme un lecteur normal mais mappe les informations de chaque ligne à un dictionnaire où les clés correspondent aux entêtes spécifiés dans la première ligne du fichier CSV.

Explication

L'utilisation de ``csv.DictReader`` permet de lire le fichier CSV et de convertir chaque ligne en un dictionnaire, rendant les données plus accessibles et plus faciles à manipuler, surtout si vous souhaitez accéder à des valeurs spécifiques par leur nom de champ. Pour trier ces dictionnaires, nous pouvons toujours utiliser la fonction ``sorted()``, en spécifiant une clé de tri qui fait référence à l'une des clés du dictionnaire.

Étapes

1. Lire le fichier CSV comme une liste de dictionnaires : Utiliser ``csv.DictReader`` pour lire le fichier et convertir chaque ligne en un dictionnaire basé sur l'en-tête du fichier CSV.
2. Trier les dictionnaires : Trier la liste des dictionnaires obtenue en utilisant la fonction ``sorted()`` et en spécifiant une clé de tri.
3. Afficher ou enregistrer les données triées : Les données triées peuvent être affichées directement ou sauvegardées dans un nouveau fichier, selon les besoins.

```
# Ouvrir et lire le fichier CSV comme une liste de #dictionnaires

with open('donnees.csv', 'r') as fichier:

    dict_lecteur = csv.DictReader(fichier)

# Conversion en liste pour le traitement

    liste_dictionnaires = list(dict_lecteur)

# Trier les dictionnaires par âge

donnees_triees = sorted(liste_dictionnaires, key=lambda x: int(x['Age']))

# Afficher les résultats

for dictionnaire in donnees_triees:

    print(dictionnaire)
```

Utilisation de Pandas pour le Tri

- Pandas simplifie grandement le tri de fichiers CSV grâce à ses fonctions intégrées, comme ``sort_values``.
- On peut trier selon une ou plusieurs colonnes, dans l'ordre croissant ou décroissant.
- Un exemple montre comment lire un fichier CSV avec pandas, trier les données selon certaines colonnes, et sauvegarder le résultat dans un nouveau fichier CSV.

Avec Pandas, nous pouvons charger un fichier CSV directement dans un DataFrame, qui est une structure de données bidimensionnelle avec des lignes et des colonnes, similaire à une feuille de calcul. Chaque colonne dans un DataFrame a un label (nom), ce qui facilite l'accès et le tri des données basé sur ces labels. Pour trier les données, nous utilisons la méthode `.sort_values()` de Pandas, qui permet de spécifier une ou plusieurs colonnes comme critères de tri.

Étapes

1. Charger le fichier CSV dans un DataFrame : Utiliser `pd.read_csv()` pour lire le fichier CSV et le charger dans un DataFrame.
2. Trier le DataFrame : Utiliser `.sort_values()` pour trier le DataFrame selon le(s) critère(s) souhaité(s), par exemple, par âge.
3. Afficher ou sauvegarder le DataFrame trié : Afficher le résultat directement dans le notebook ou l'enregistrer dans un nouveau fichier CSV avec `.to_csv()`.

Supposons que nous voulions trier le fichier CSV par âge. Voici comment nous pourrions procéder :

```
import pandas as pd

# Charger le fichier CSV dans un DataFrame

df = pd.read_csv('donnees.csv')
```

```
# Trier le DataFrame par âge

df_trie = df.sort_values(by='Age')

# Afficher les résultats

print(df_trie)

# Optionnel : Sauvegarder le DataFrame trié dans un nouveau fichier CSV

# df_trie.to_csv('donnees_triees.csv', index=False)
```

Avec Pandas, vous pouvez trier un DataFrame par plusieurs colonnes. La méthode `sort_values()` prend en charge le tri par plusieurs colonnes en spécifiant une liste des noms de colonnes à trier par ordre de priorité. Voici comment vous pourriez procéder :

Supposons que vous vouliez trier d'abord par l'âge, puis par le nom. Voici comment vous pouvez le faire :

```
import pandas as pd

# Charger le fichier CSV dans un DataFrame

df = pd.read_csv('donnees.csv')

# Trier le DataFrame par âge, puis par nom

df_trie = df.sort_values(by=['Age', 'Nom'])

# Afficher les résultats

print(df_trie)

# Optionnel : Sauvegarder le DataFrame trié dans un nouveau fichier CSV

# df_trie.to_csv('donnees_triees.csv', index=False)
```

- ⇒ Dans cet exemple, `by=['Age', 'Nom']` spécifie que le tri se fait d'abord par la colonne 'Age', puis par la colonne 'Nom'. Vous pouvez spécifier autant de colonnes de tri que nécessaire en les ajoutant à la liste passée à `by`.
- ⇒ Le DataFrame sera trié en fonction de la première colonne spécifiée ('Age') et, en cas d'égalité, les valeurs seront triées selon la deuxième colonne spécifiée ('Nom').

PS : Pour trier une liste de dictionnaires (simulant des données CSV) par deux colonnes sans utiliser Pandas, vous pouvez en effet utiliser une expression lambda avec la fonction `sorted()`.

L'expression lambda retournera un tuple composé des éléments des colonnes par lesquelles vous souhaitez trier. Cela indique à `sorted()` de d'abord trier par le premier élément du tuple, puis par le second en cas d'égalité.

Voici un exemple montrant comment trier par deux colonnes, disons d'abord par "Age" puis par "Nom" :

```
donnees = [  
    {'Nom': 'Alice', 'Age': 25, 'Ville': 'Paris'},  
    {'Nom': 'Bob', 'Age': 30, 'Ville': 'New York'},  
    {'Nom': 'Claire', 'Age': 28, 'Ville': 'Sydney'},  
    {'Nom': 'Alice', 'Age': 22, 'Ville': 'Londres'} # Ajouté pour illustrer le tri  
]  
  
# Trier par Age, puis par Nom  
  
donnees_triees = sorted(donnees, key=lambda x: (x['Age'], x['Nom']))  
  
for d in donnees_triees:  
    print(d)
```

Dans cet exemple, `x['Age'], x['Nom']` spécifie que les données doivent d'abord être triées par l'âge. Si deux personnes ont le même âge, le tri secondaire par nom est appliqué. Le résultat est une liste de dictionnaires triée d'abord par l'âge en ordre croissant, puis par le nom en ordre alphabétique en cas d'âges identiques.

Cette approche est très flexible et peut être adaptée pour trier par n'importe quelles colonnes en ajustant simplement les clés utilisées dans l'expression lambda.

Cette leçon illustre l'importance de savoir trier des données, une compétence fondamentale en informatique, et montre plusieurs méthodes pour y parvenir efficacement en Python.